SIT 218/738: Secure coding

Pass task 3.1P: Injection attacks-Part 1

Objective

In this task you will insert a script into the vulnerable web application and observe the impact. Furthermore, you will analyse the code and understand the vulnerability in the web application.

Overview

Task 1: Complete the steps provided in "**Ontrack Task3.1P Start Activity"** on CloudDeakin before completing this task.

You need to include screenshots of activities (1 to 3) and outputs in your submission.

Task 2:

Continuing from the Ontrack Task3.1P start activity with the **02-vul-coachwebapp,** insert the below scripts into the name field and view the result in the browser.

<form><button form onclick=javascript:alert(1)>CLICKME

<form><button formaction=javascript:alert(1)>CLICKME

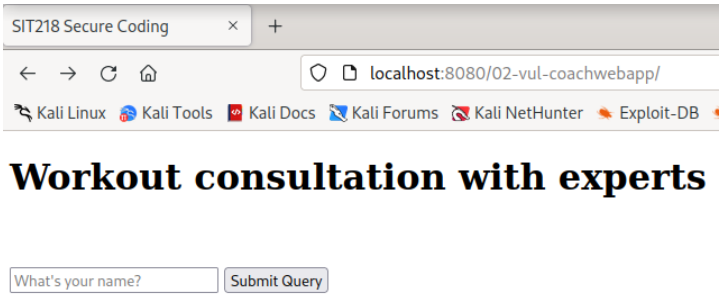
Note: in the above script "formaction" is one word.

Record the outputs generated. What is its impact on the outputs of the webapp. Explain briefly the reasons for this attack to be successful.

Task 3: Reflected XSS

Continuing from the Ontrack Task3.1P start activity with the **02-vul-coachwebapp,**

Once the application executes you have the following page:



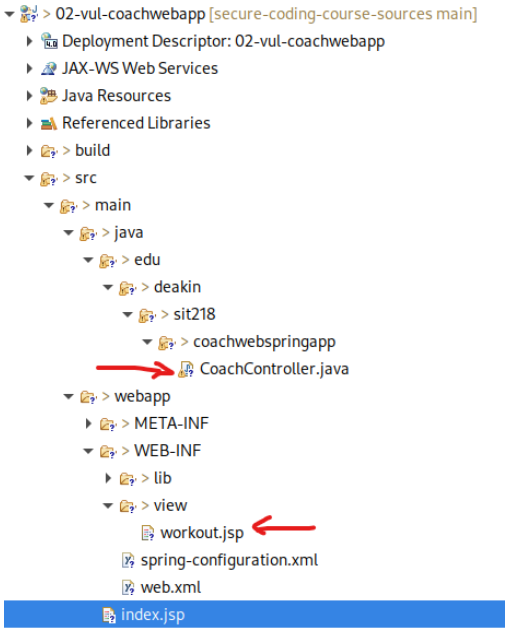
Inject the following into the name field: <script>alert('Reflected XSS')</script>

a. Note whether the alert executes and explain why.

b. Did the processing of your input occur on the server side or client (browser) side?

Task 4: DOM based XSS

In the 02-vul-coachwebapp make the following changes



Comment out the following line in your **CoachController.java** class as below:

//model.addAttribute("name", name);

```
@GetMapping
public String workout(@RequestParam("studentName") String name,

                      Model model) {

//model.addAttribute("name", name);


            Random r = new Random();
    if (r.nextBoolean())
        {model.addAttribute("message", "No training today");

        }
```

In the **workout.jsp** under the view folder, delete the line which is highlighted in yellow and add the lines marked between 12 and 22 including line 13:

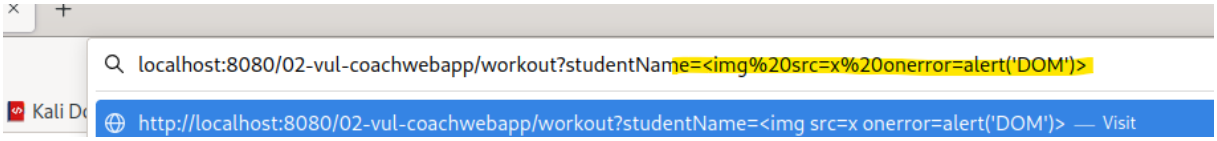
```
1  <%@ page language="java" contentType="text/html; charset=ISO-8859-1"  pageEncoding="ISO-8859-1"%>
2  <!DOCTYPE html>
3  <html>
4  <head>
5  <meta charset="ISO-8859-1">
6  <title>First spring webcoach-app</title>
7  </head>
8  <body>
9  <h1>Coach's Message</h1>
10
11 <p id="p1">${name}</p>
12
13 <div id="output"></div>
14
15 <script>
16
17 var name = new URLSearchParams(window.location.search).get("studentName");
18
19 document.getElementById("output").innerHTML = "Hello " + name ;
20
21 </script>
22
23 <p id="p1">${message}</p>
24
25 </body>
26 </html>
```

Your updated code should look like

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"  pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>First spring webcoach-app</title>
</head>
<body>
<h1>Coach's Message</h1>

<div id="output"></div>

<script>

var name = new URLSearchParams(window.location.search).get("studentName");

document.getElementById("output").innerHTML = "Hello " + name ;

</script>

<p id="p1">${message}</p>

</body>
</html>
```

Now save the CoachController.java class and workout.jsp code

Now inject: <img src=x onerror=alert('DOM')> this into the name field and click Submit Query. Once the workout.jsp page loads enter the injection script in the URL as shown below:



Capture the response and address the following questions:

a. What differences did you observe between the first and second injection attempts?

b. What was the source of the input that triggered the alert?

c. Was your input processed on the server side or in the browser (client side)?

d. In the case of DOM-based XSS, was the injected script reflected back to the client side?

e. Compare the location of the vulnerability in both the reflected and DOM-based XSS attempts. Did it occur in the CoachController class (processed on the server) or in the workout.jsp page (executed in the client browser)?

f. Can static HTML pages be vulnerable to DOM-based XSS attacks? Provide a brief explanation.

Task 5:

Study about different types of cross site scripting attack (XSS from the CloudDeakin unit site week 3, Book “Cross-Site Scripting Attacks : Classification, Attack, and Countermeasures, Author B. B. Gupta and Pooja Chaudhary” chapter-3 “Fundamentals of Cross-Site Scripting (XSS) Attack” and following website from Mitre CAPEC - CAPEC-63: Cross-Site Scripting (XSS) (Version 3.9) (mitre.org).

Prepare a two-page report on different types of Cross-Site Scripting (XSS) Attack. Your report should include description of XSS attacks, their effects and description on mitigation techniques.

Submission Requirements:

Submit one PDF file containing the following information:

1. Screenshots of activities (1 to 3) and outputs from **Ontrack Task3.1P Start Activity**
2. Screenshots of the output generated by the web browser.
3. URL with injected code
4. Brief explanation of the vulnerability in **02-vul-coachwebapp** highlighting the vulnerable piece of code. You can access the code in the Eclipse IDE in the provided Kali VM.
5. Answer of Task 2
6. Screenshots and answers of questions in Task 3 and Task 4
7. Answer of Task 5

Submission Due

The due for each task has been stated via its OnTrack task information dashboard.